UNIVERSITÀ DI PARMA

Predefined Functions

E Pluribus Unum

Autore Ignoto, Moretum, I sec. a.C.

Sommario

UNIVERSITÀ DI PARMA

- Why we need functions?
- C standard library
- Predefined functions
  - Anatomy of a function

Definition of a function

UNIVERSITÀ DI PARMA

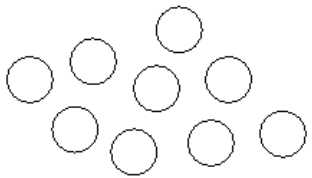
- A function is a **group of statements** that, together, perform a given task
  - i.e. performing something **more complex** than "simple" C instructions
  - All high level programming languages have the concept of "function"
  - Every C program features at least one function → main()
- In C we also have macros, but we won't talk about them

Why we want to use them?

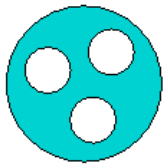
UNIVERSITÀ DI PARMA

- **Factorization:** a function is defined once but can be used countless
  - Reusable code
  - Reduce code size ⟶ greater efficiency
- **Ease of modification:** redundancy is reduced
  - I modify/improve/correct code in one place
- **Better readability:** code details can be isolated
  - The use of functions can be self-explanatory
  - Breaking the code in smaller functions, makes it more readable and structured

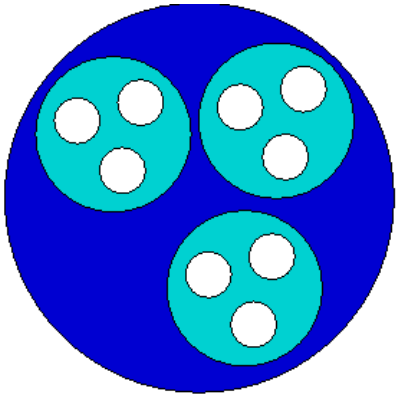
Software levels



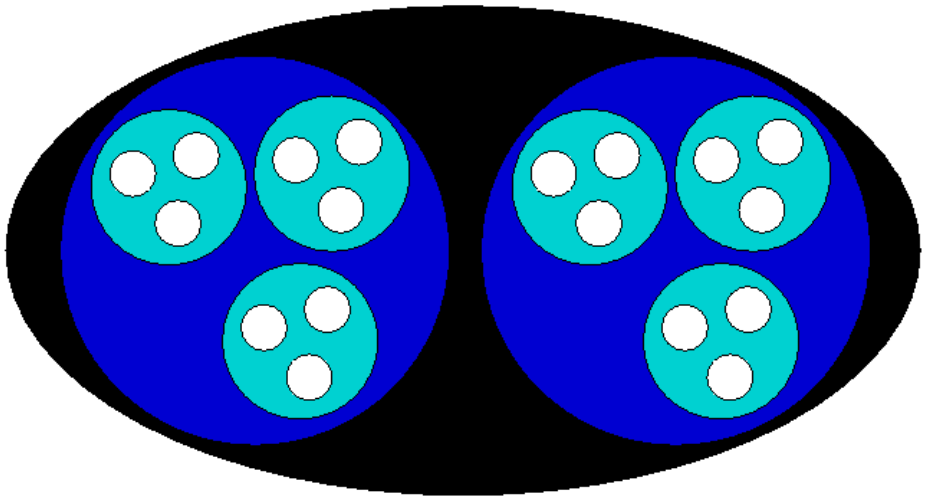
Machine Instructions



Programming Language Instructions

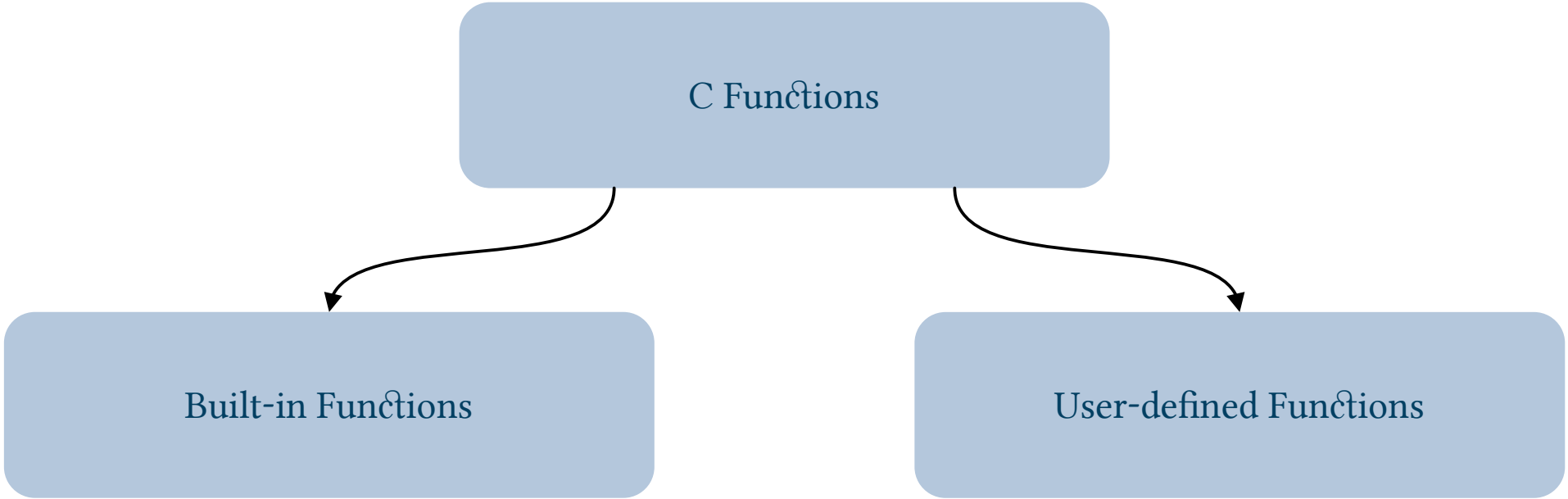


Functions



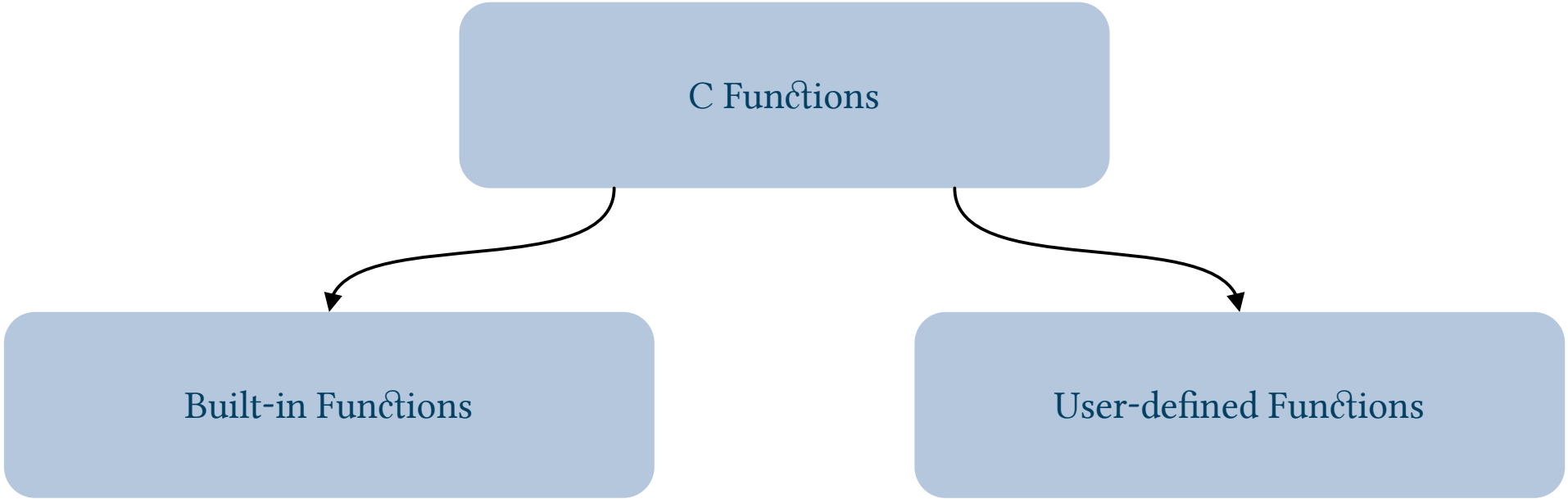
Program

Two types of functions




```
printf()
scanf()
malloc()
rand()
...
```

Two types of functions




```
printf()
scanf()
malloc()
rand()
...
```

Built in functions/library functions

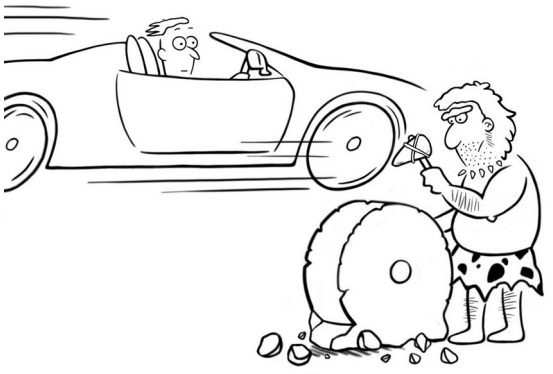
UNIVERSITÀ DI PARMA

- Function already available that are provided by the C library
  - Input/output: printf(), scanf()...
  - Math: sqrt(), cos(), sin()...
  - Time/Dates: time()...
  - Memory management: not yet discussed
  - String/Character: not yet discussed
  - Other: rand(), srand()...
  - ...

Underlying idea

UNIVERSITÀ DI PARMA

- Do not reinvent the wheel!
- The *C Standard Library* enable us with a lot of useful functions
- Anyway
  - We need to know what functions are provided…
  - Include a specific header
  - Use them!
    - This also implies we need to know how to use them



Main C header files

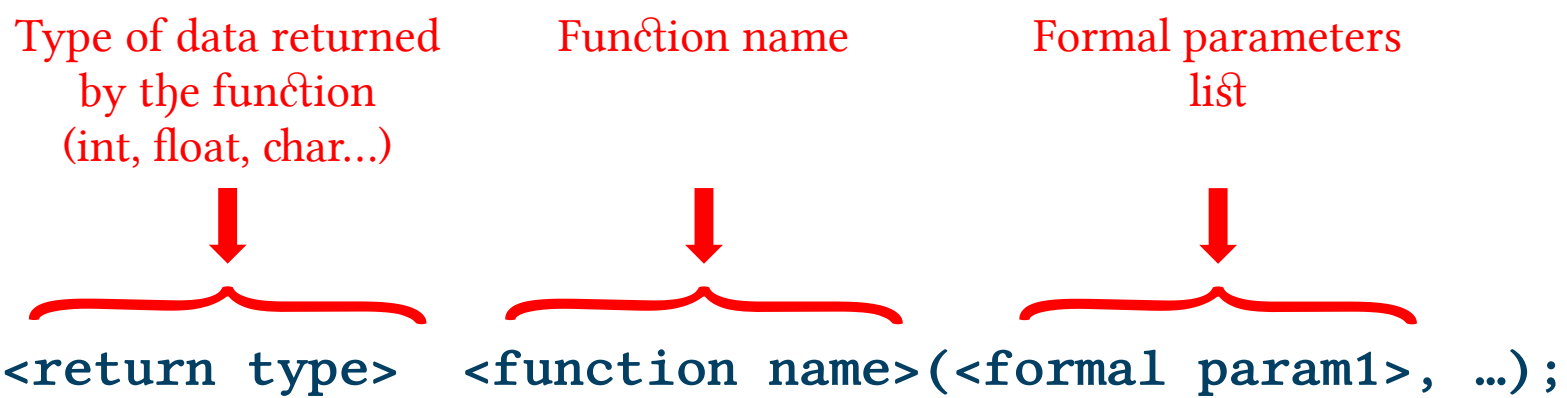
UNIVERSITÀ DI PARMA

- `stdlib.h` → general use (I/O, memory, data types...)
- `string.h` → string management
- `math.h` → mathematical functions
- `time.h` → time and calendar management
- `ctype.h` → characters inspection and management

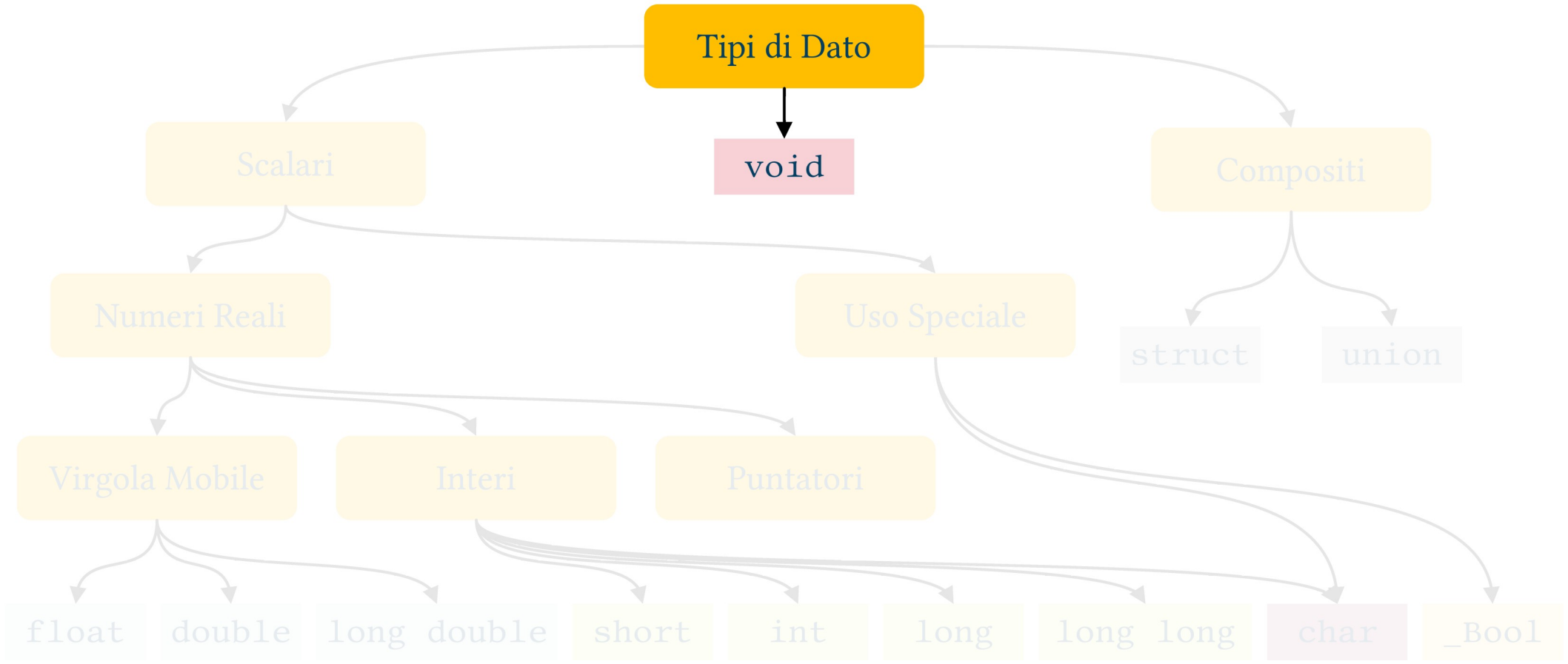
Function "signature"

UNIVERSITÀ DI PARMA

- When we look at any C manual we can find a function description
- and a function "signature", something like:
  - int rand(void); void srand(unsigned int seed); double sqrt(double x);



C99 Albero dei Tipi di Dato



void

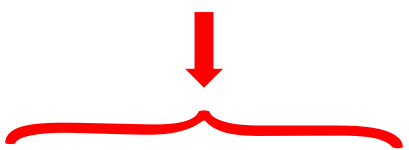
- It is a *no-type*
- We can not use *void* to define variables
- Used for
  - Functions
    - No value
    - No parameters
  - Pointers (we will see them later, spoiler: initially you will not like them)
    - No type

Function name

UNIVERSITÀ DI PARMA

- What we need to "use" it
- Again remember that lower case ≠ upper case

Function name

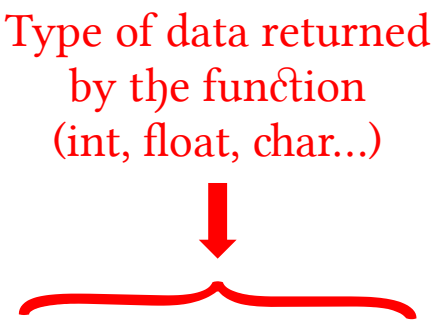


```
<return type>  <function name>(<formal param1>, …);
```

Returned type

UNIVERSITÀ DI PARMA

- It allows us to understand which type of data the function provides
  - int rand(), double sqrt(), **int** abs()
- Not all functions return something
  - void srand()

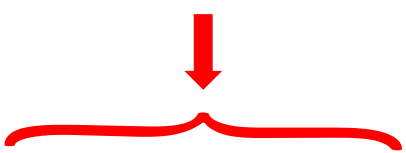



```
<return type>  <function name>(<formal param1>, …);
```

List of formal parameters

UNIVERSITÀ DI PARMA

- Many functions needs some data as input
  - double sqrt(double x);
- Sometimes more than one parameter ⟶ comma is used
  - double pow(double x, double y);
- Or even nothing
  - int rand(void);




```
<return type>  <function name>(<formal param1>, …);
```

- A good manual also point us which header file is needed

exit *Exit from Program* `<stdlib.h>`

```
void exit(int status);
```

Calls all functions registered with `atexit`, flushes all output buffers, closes all open streams, removes any files created by `tmpfile`, and terminates the program. The value of `status` indicates whether the program terminated normally. The only portable values for `status` are `0` and `EXIT_SUCCESS` (both indicate successful termination) plus `EXIT_FAILURE` (unsuccessful termination).

9.5, 26.2

Invoking a pre-defined function

UNIVERSITÀ DI PARMA

- The knowledge of the function signature is all what we need

```
x1 = (-b + sqrt( pow(b,2) - 4*a*c )/( 2*a );
x2 = (-b - sqrt( pow(b,2) - 4*a*c )/( 2*a );
```

UNIVERSITÀ DI PARMA

Predefined Functions

E Pluribus Unum

Autore Ignoto, Moretum, I sec. a.C.